//Alchemy Bare SMA7702 PR Review

August 5 2026

About Octane

Founded in 2023 and headquartered in San Francisco, Octane delivers a truly holistic security solution through continuous code reviews that protect protocols throughout their entire development lifecycle. We combine software-first vulnerability analysis with world-class manual security talent, enabling teams to ship faster while maintaining the highest security standards. Our clients including Circle (USDC), Uniswap, Avalanche, Plume, and Sophon trust us to protect $5.8B+ in assets because we catch vulnerabilities that traditional assessments miss.

Clients choose Octane for:

- Our software-first approach: We've built a platform trained on thousands of real-world vulnerabilities that consistently identifies complex exploit patterns traditional methods miss. Every commit and pull request receives automatic review through native CI/CD integration, with vulnerabilities identified through deep dependency analysis and exploit path tracing.

- Our novel research: Octane's security researchers who've collectively earned over $1M in bug bounties provide comprehensive manual reviews on an as-needed basis, maximizing code security and developing further insights for automated protection.

- Our proven track record preventing exploits: We've discovered 100+ vulnerabilities across 86M+ lines of code that were missed in traditional security reviews. We prevented exploits similar to Rikkei Finance ($1.1M), caught complex slippage bugs in Uniswap integrations, identified oracle mispricing vulnerabilities, and found assembly errors that evaded static analyzers. Our platform adapts to your team's preferences, dramatically reducing false positives as the platform learns your codebase while providing exact code modifications and exploit scenarios for immediate remediation.

- Our commitment to ecosystem security: We partner directly with blockchain foundations to establish network-wide security baselines. Integration requires no code changes and takes minutes to deploy, after which teams receive continuous protection with rapid response. We offer flexible engagement from basic reviews to enterprise coverage with 24/7 support and formal security reports, supporting all EVM-compatible chains with developing Rust capabilities.

Through our holistic approach, we transform security from a development bottleneck into an enabler of velocity and confidence. We live by iterative development and in August FY25 alone, we boosted accuracy on challenging vulnerability classes by 31% and increased our detection coverage by 12.7%. To learn how continuous security accelerates development while protecting users, visit octane.security.

Executive Summary

`SemiModularAccount7702` adds ERC-1271 compatibility for bare externally owned account (EOA) signatures. An EIP-7702 delegated EOA can validate canonical 64-byte ERC-2098 and 65-byte ECDSA signatures without modular-signature framing.

The bare-signature path behaved as designed in the reviewed scope. It is active only when fallback signing is enabled, the resolved fallback signer is the delegated account, and native fallback validation has no pre-validation hooks. A new, otherwise unconfigured SMA7702 meets these conditions by default. In raw mode, a canonical signature that recovers to the delegated EOA returns the ERC-1271 success value. Malformed and non-matching 64- or 65-byte inputs return `0xffffffff` without reverting. This path neither writes account storage nor invokes modular validators.

The review identified one Low-severity finding and one Informational finding. The Low-severity issue concerns lifecycle ordering: installation callbacks can use validator or selector authority before later restrictive hooks are installed, while uninstall hook callbacks run before that authority is cleared. The module must already have the relevant authority; installation as a hook alone does not grant it. The Informational issue concerns short-circuit callback aggregation: one failed uninstall callback skips later callbacks even though account metadata removal continues.

No other findings were identified in the bare-signature implementation or the tested integrations.

Two properties of the design affect how integrations validate and revoke signatures:

- A hook can disable bare-signature ERC-1271 validation, but it cannot universally revoke the EOA signature. An ECDSA-first verifier may validate the signature without calling the account. The repository's OpenZeppelin version does this for 65-byte signatures; its Solady version does it for both 64- and 65-byte signatures.
- Bare signatures cover the application-supplied digest directly. The consuming application must bind the digest to the intended chain, protocol, signer, action, amount, nonce, and deadline, just as it would for an ordinary EOA signature.

Findings Summary

ID	Title
L-01	Lifecycle callbacks execute before the final validation policy is in force
I-01	A failed uninstall callback skips later callbacks

2. Scope and Coverage

The review assessed the changes introduced by `feat/sma-1.1.0` at commit `0a78d41`, relative to the `v2.0.x` baseline at commit `c9e7683`, in `alchemyplatform/modular-account-octane`. The review covered the SMA7702 bare-signature ERC-1271 entry point and the account state and integrations that control or consume it:

- raw-mode activation and deactivation;
- fallback signer rotation and fallback disablement;
- canonical 64-byte and 65-byte ECDSA recovery;
- malformed signatures and storage immutability;
- hook installation, uninstallation, duplicates, callback failures, count/list consistency, and maximum cardinality;
- modular signature framing at the reserved lengths;
- lifecycle callback reentrancy;
- delegation replacement and persistent account storage;
- replay and migration consequences;
- Permit2-style contract-signature dispatch;
- OpenZeppelin and Solady caller behavior;
- deployed-signer, unwrap-only ERC-6492 handling;
- stale validation results and point-of-use revalidation; and
- separation from UserOperation, runtime, deferred-action, and non-7702 SMA validation paths.

3. Feature Overview

Bare signatures describe the ERC-1271 input and dispatch path. Raw mode describes the account state in which SMA7702 interprets an exact 64- or 65-byte input as bare ECDSA instead of modular-signature framing.

3.1 Bare-signature ERC-1271 path

`SemiModularAccount7702.isValidSignature` reserves exact 64- and 65-byte inputs for bare ECDSA while raw mode is active. If raw mode is inactive, the same bytes enter ordinary modular ERC-1271 decoding.

3.2 Raw-mode activation and default state

Raw mode is active only when all of the following are true:

1. fallback signing is enabled;
2. the resolved fallback signer equals `address(this)`; and
3. `FALLBACK_VALIDATION` has no pre-validation hooks.

The implementation reads these predicates at `src/account/SemiModularAccount7702.sol:61-73`. The zero storage value for `fallbackSigner` resolves to `address(this)` on SMA7702, so a new, otherwise unconfigured account starts in raw mode. Storing the account address explicitly has the same effect. Storing a different EOA or contract, or setting `fallbackSignerDisabled`, deactivates raw mode.

Execution hooks, hooks associated with another validation key, and validation flags do not control raw mode. A pre-validation hook associated specifically with native fallback validation does.

3.3 Signature recovery and return behavior

The bare-signature branch accepts:

- 65-byte `r || s || v` signatures, with `v` equal to 27 or 28; and
- 64-byte ERC-2098 `r || vs` signatures.

Recovery uses OpenZeppelin `ECDSA.tryRecover`, which enforces canonical low-`s` signatures. A signature succeeds only when recovery returns `address(this)`. Recovery

failure, a non-canonical signature, an invalid `v`, the wrong key, or the wrong digest returns `0xffffffff`.

For exact 64- and 65-byte inputs while raw mode is active, recovery does not revert, invoke a modular validator, invoke pre-validation hooks, or mutate account storage.

3.4 Separation from other authorization paths

The bare-length dispatcher exists only in public ERC-1271 validation. It does not reinterpret bare signatures in:

- UserOperation validation;
- runtime validation;
- deferred validation installation; or
- other SMA variants.

The native fallback path rejects `CONTRACT_OWNER` when the resolved fallback signer is the account itself. This prevents direct self-reference through ERC-1271 from promoting a signature-only validation into fallback-global UserOperation authority. A distinct contract fallback signer remains supported.

4. Findings

4.1 L-01: Lifecycle callbacks execute before the final validation policy is in force

Severity: Low

Description

Validation installation and uninstallation call external modules while account authorization is in an intermediate state.

During installation, `ModuleManagerInternals._installValidation`:

1. writes the validation module, flags, and selectors;
2. inserts one hook;
3. calls that hook's `onInstall`; and
4. repeats for the remaining hooks, then calls the validation module's `onInstall`.

An early hook callback can see the validator and earlier hooks, but not later restrictive hooks. Direct-call selectors are also active before later hook callbacks run.

During uninstallation, the account calls hook `onUninstall` functions before clearing the hooks, selectors, flags, counts, and validation function. It calls the validation module's `onUninstall` after clearing. Hook callbacks run while the authority being removed is still active.

Tests demonstrated the following behavior:

- an early installation callback validated a signature before a later hook could reject it;
- a module with direct-call authority for `installValidation` installed another validation before a later restrictive hook was added;
- the later hook blocked the same operation after installation completed;
- an uninstall hook used still-active authority for a nested account-management call;
- recursive uninstall removed the validation without restoring or corrupting it; and
- the validation module's post-clear callback could not use the removed authority.

Installation as a hook does not grant account-management authority. Nested mutation failed without direct-call authorization. This issue requires an installation that explicitly grants the callback module authority that the final hook configuration is meant to constrain.

Impact

Integrators may assume that the complete hook policy protects every installation callback, or that uninstall callbacks run only after authority is revoked. A malicious or compromised authorized module can instead act during these intermediate states.

The module can perform account mutations or validations that the final policy would reject. Its actions remain limited to authority granted during installation; arbitrary external callers gain no new privileges.

Recommendation

Commit the complete validation and hook configuration before installation callbacks can use it. Revoke authority before uninstallation callbacks run. If the module standard requires the current order, define lifecycle callbacks as trusted execution that later hooks do not protect.

At minimum:

- document that later hooks do not protect earlier `onInstall` callbacks;
- document that hook `onUninstall` callbacks run while validation authority is active;
- do not grant account-management selectors to modules with untrusted lifecycle callbacks;

4.2 I-01: A failed uninstall callback skips later callbacks

Severity: Informational

Description

Account metadata removal continues after an uninstall callback fails. The implementation combines callback results with short-circuiting `&&`, so it does not attempt later hook callbacks or the validation module callback.

Impact

Account validation metadata is still cleared, but later modules may miss per-account cleanup. Modules must not rely on `onUninstall` for security-critical cleanup.

Recommendation

Treat `onUninstall` as a best-effort notification, not a security-critical cleanup mechanism. To notify every module, call each callback independently and aggregate the results afterward. If callback behavior cannot change, document that a failed callback skips the rest and prohibit security-critical `onUninstall` cleanup.

5. Security analysis and integration considerations

5.1 Raw-mode invariant and hook state

Stateful tests added or substantially expanded during this review combined signer rotation, fallback enablement and disablement, hook changes, duplicate and reverting operations, unrelated modules, and delegated-code replacement.

A valid self-key signature returned the ERC-1271 success value if and only if all activation predicates held and canonical ECDSA recovery matched the account. In raw mode, malformed 64- and 65-byte values returned exactly `0xffffffff` without reverting. These calls did not write storage or invoke modular validation.

Raw-mode eligibility reads `validationHookCount` from native fallback validation. Duplicate hooks and reverting installation callbacks reverted the full transaction, including counter and linked-list updates. No count/list divergence occurred.

Uninstallation cleared hooks, selectors, flags, module data, and both hook counts. A reverting callback did not stop metadata removal. Clearing restored raw mode when the other activation predicates held.

Installation, removal, and reuse succeeded at the configured maximum hook count. One-over-limit and duplicate attempts reverted atomically. These boundary operations were gas-intensive but did not corrupt state. Deployment tooling should reject an invalid hook count before submitting the transaction.

With enough outer gas, the account caught callbacks that exhausted their forwarded gas or reverted with 32 KiB of data. It then cleared metadata and restored raw mode. If the outer call lacked enough gas to finish clearing, the full uninstall reverted. The validation stayed installed and raw mode stayed disabled. A later retry with enough gas succeeded. This is a liveness constraint, not partial state corruption.

`getValidationData()` reverses the loaded hook array for display. Uninstallation instead iterates the underlying linked list. As a result, `hookUninstallDatas` must use the reverse of the public hook-view order. Tests with two hooks confirmed this mapping. SDK and API documentation should state the order so callers do not send uninstall data to the wrong module.

5.2 Replay, migration, and digest binding

Bare signatures cover the caller-supplied digest directly. The account does not add a chain- or account-scoped domain.

The consuming application must bind the digest to the intended chain, verifying contract, signer, action, amount, nonce, deadline, and protocol-specific context. This matches ordinary EOA semantics. If the application separates its digest poorly, the same signature may be actionable in another context. This risk is not unique to delegated accounts or ERC-1271; the compatibility path accepts the approval produced by the EOA key.

A canonical signature created:

- before delegation;
- while raw mode was disabled;
- before replacing delegated code; or
- under an earlier implementation that did not accept bare ERC-1271 signatures

can later validate when raw mode becomes active, as long as the application's nonce, deadline, and revocation state still permit it.

A fallback hook or a rotated or disabled fallback signer makes the signature dormant for code-first ERC-1271 consumers. Removing the hook, restoring the self signer, or replacing compatible delegated code can make it valid again. Disabling raw mode is reversible configuration, not cryptographic revocation. Before a migration activates bare-signature compatibility, invalidate any live application approval that should not survive the migration.

Permit2 served only as an example of a code-first ERC-1271 consumer. The test dispatcher forwards an application-supplied digest and signature to ERC-1271 when the signer has code. The review did not independently verify Permit2's typed-data construction or the fields bound by each permit variant. Integrators must check replay, spender, recipient, amount, nonce, deadline, witness, chain, and verifying-contract binding for the exact Permit2 function and deployed version they use.

5.3 Exact-length compatibility boundary

While raw mode is active, every 64- or 65-byte value is interpreted exclusively as bare ECDSA. A modular signature with either total length is shadowed and does not reach its validation module.

The modular ERC-1271 encoding length is:

```
None
locator prefix
+ sparse hook segments
+ final-segment marker
+ module signature
```

An entity locator contributes five bytes, a direct-call locator contributes 21 bytes, each supplied sparse hook segment contributes five framing bytes plus its non-empty data, and the final marker contributes one byte.

Without hook data, the colliding module payload lengths are:

Locator	Payload for total 64	Payload for total 65
Entity	58 bytes	59 bytes
Direct call	42 bytes	43 bytes

Boundary tests added during this review covered entity and direct-call locators, up to three sparse hook segments, varied hook data, and both reserved totals. In raw mode, the account returned `0xffffffff` without calling a validator configured to revert. After signer rotation disabled raw mode, the same bytes reached that validator through modular decoding.

The supported built-in formats do not collide. A SingleSigner EOA payload produces a 72-byte entity-locator ERC-1271 signature. WebAuthn signatures and multisig UserOperation aggregates are larger.

Custom variable-length modules can still produce a complete 64- or 65-byte encoding. They must pad or otherwise change that encoding in raw mode. `ValidationLocatorLib.packSignature` does not reject these totals. SDKs should warn when they produce one, and custom validation modules should document their possible signature lengths and collision-avoidance behavior.

ERC-6492 adds no separate outer-length collision in the tested deployed-signer path. After unwrapping, bare and modular 64- and 65-byte signatures followed the same raw-mode rules.

5.4 Consumer interoperability

Installing a hook on native fallback validation disables the account's bare-signature ERC-1271 branch, but it does not prevent a verifier from validating the EOA signature independently.

Consumer harnesses added during this review established the following behavior:

Consumer behavior	64-byte signature after hook installation	65-byte signature after hook installation
Repository OpenZeppelin, ECDSA-first	Falls through to ERC-1271 and is rejected	Recovered directly and remains valid
Repository Solady, ECDSA-first	Recovered directly and remains valid	Recovered directly and remains valid
Code-first verifier	Sent to ERC-1271 and rejected	Sent to ERC-1271 and rejected
Permit2-style direct ERC-1271	Rejected	Rejected

Library versions can change this behavior. Check the deployed verifier instead of assuming every `SignatureChecker` dispatches the same way. A fallback validation hook is an ERC-1271 mode switch, not universal revocation of the EOA signature. It also cannot restrict top-level transactions sent by the delegated EOA.

In raw mode, both `STATICCALL` and `CALL` receive a canonical 32-byte ABI result. Callers should prefer `STATICCALL` because it enforces read-only behavior.

Outside raw mode, a bare 64- or 65-byte signature enters modular decoding and may revert instead of returning `0xffffffff`. Consumers that map ERC-1271 call failure to `false` handle this case cleanly. Consumers that propagate the revert can turn an invalid signature into an application-level revert or estimation failure.

SMA7702 works with strict callers that require a successful call and a canonical 32-byte ABI value. First-four-byte parsing is more permissive but provides no needed compatibility for this implementation. A caller-imposed gas cap below validation cost creates false negatives. Provide enough gas rather than relying on a narrow measured threshold.

ERC-1271 validity depends on mutable signer, hook, and delegation state. Do not treat a positive or negative result as timeless. A cached valid result remained positive after signer rotation or hook installation. Cache-only execution accepted that stale decision; point-of-use validation rejected the signature and left the request unconsumed.

Validate at the point of use. If caching is unavoidable, bind the entry to an exact block or state context and account for changes earlier in the same transaction.

5.5 Storage and redelegation behavior

The bare-signature change adds dispatch logic but no fields to the current modular-account or fallback-signer storage structures.

Tests replaced delegated runtime code with another build using the reviewed layout. The replacement preserved fallback signer state, validation flags and selectors, hook counts and lists, and raw-mode eligibility. Public validation metadata encoded to the same value before and after replacement.

These tests confirm persistence across replacements that use the reviewed layout.

6. Verification

Verification combined the feature branch's existing SMA7702 regression suite with adversarial tests added or expanded during this review. All focused suites and the existing regression suite passed.

6.1 Existing regression coverage

The existing suite covered the feature's baseline ERC-1271 behavior and broader SMA7702 account functionality. It was retained as a regression check while the review added focused adversarial coverage around the feature's state, lifecycle, and integration boundaries.

6.2 Verification added during this review

The review added focused test and mock files and substantially expanded the central SMA7702 test suite. Added or expanded coverage included:

- the raw-mode if-and-only-if invariant across arbitrary signer, fallback, hook, unrelated-module, and delegated-code transitions;
- canonical, malformed, and non-matching 64- and 65-byte signatures, including exact `0xffffffff` failure behavior and storage immutability;
- exact-length modular-signature collisions across entity and direct-call locators, sparse hook framing, and both reserved totals;
- hook installation and uninstallation atomicity, count/list consistency, callback failure, persistent metadata, maximum cardinality, and reuse;
- lifecycle callback reentrancy against intermediate installation and uninstallation authority;
- gas-exhausting callbacks, 32 KiB revert data, outer-gas rollback, and successful retry;
- ERC-1271 `STATICCALL` and `CALL`, return-data parsing, revert normalization, and caller-imposed gas caps;
- ECDSA-first, code-first, and direct ERC-1271 consumer dispatch;
- deployed-signer ERC-6492 unwrapping and inner-signature routing; and
- stale cached validity after signer or hook changes, compared with point-of-use validation.

These tests established that activation follows the three raw-mode predicates; canonical self-key signatures return the ERC-1271 success value; malformed or non-matching exact-length inputs return `0xffffffff` in raw mode; bare-signature validation does not write storage or invoke modular validation; hook metadata remains consistent across tested boundary and failure cases; and redelegation preserves state when the replacement uses the reviewed layout.